

Reviewer Pack — Hodge Diamond Rank of Boolean Algebras vs AC^0 PARITY Depth

Entry 9d4e7ed1496a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 00:20:13 UTC

Hodge Diamond Rank of Boolean Algebras vs AC^0 PARITY Depth

Entry ID: 9d4e7ed1496a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 00:20:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: AC^0 Circuit Complexity

Statement:

[‘For every boolean algebra B with n generators and k relations, the Hodge diamond rank of its associated variety $V(B)$ is upper bounded by a function $f(n, k)$ such that the AC^0 PARITY depth of any circuit computing B is at least $\Omega(f(n, k))$.’, ‘Equivalently, for any boolean algebra B and its associated variety $V(B)$, the number of Hodge diamond dimensions in $V(B)$ is proportional to the complexity of an AC^0 PARITY circuit computing B .’, ‘Formally, for all $n \geq 1$ and $k \geq 0$, if C is an AC^0 PARITY circuit with n inputs and depth $d(C) \leq d$, then $\dim(\text{HodgeDiamond}(V(B))) = O(n^d)$.’]

Rationale (proposer’s reasoning):

[‘The Hodge diamond of a variety encodes information about its algebraic structure, which may reflect the complexity of computing properties of the associated boolean algebra.’, ‘This conjecture suggests that the geometric complexity of the Hodge diamond could be used to infer circuit complexity bounds for boolean algebras.’, ‘If true, this would provide an alternative perspective on AC^0 PARITY depth by relating it to algebraic geometry.’]

Taxonomy category: AC^0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 18bf85d28c909952

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the given boolean algebra B with n generators and k relations, the Hodge diamond rank of $V(B)$ will be considered supported if the AC^0 PARITY depth of any circuit C computing B is at least $\Omega(f(n, k))$ where $f(n, k)$ is a function upper bounding the Hodge diamond rank. Falsification will occur if there exists a circuit C with n inputs and $d(C) \leq d$ such that $\dim(\text{HodgeDiamond}(V(B))) > O(n^d)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Hodge Diamond Rank AND boolean algebra AND AC0 parity - arxiv:math.AG AND arxiv:cs.CC WITH 'Hodge diamond rank' AND 'boolean algebra' - AC0 parity depth AND Hodge diamond rank AND variety V(boolean algebra)

Top relevant hits considered: - [http://arxiv.org/abs/1611.00805v3] High harmonic generation from bulk diamond driven by intense femtosecond laser pulse - [http://arxiv.org/abs/1911.04516v1] Boolean lattices in finite alternating and symmetric groups - [http://arxiv.org/abs/1712.03775v4] A Serre weight conjecture for geometric Hilbert modular forms in characteristic p - [http://arxiv.org/abs/1802.03100v1] Rank Selection and Depth Conditions for Balanced Simplicial Complexes - [http://arxiv.org/abs/1606.04200v2] The Chasm at Depth Four, and Tensor Rank : Old results, new insights - [http://arxiv.org/abs/2008.00266v1] On parity decision trees for Fourier-sparse Boolean functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_boolean_algebra(n, k):
    if n <= 0 or k < 0:
        return [], []

    elements = [tuple(sorted(random.sample(range(2), n))) for _ in range(k)]
    relations = [(i, j) for i in range(k) for j in range(i+1, k) if elements[i] != elements[j]]

    return elements, relations

def hodge_diamond_rank(elements):
    # Placeholder implementation; actual computation depends on the algebraic structure
    return len(elements)

def ac0_parity_depth(n, d):
    # Placeholder implementation; actual computation depends on the circuit complexity
    return 1 + int(Fraction(d, n).limit_denominator().numerator / Fraction(d, n).limit_denominator())

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

results = []
for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5): # Test each size with 5 instances
        k = random.randint(0, min(n, 10)) # Ensure k is non-negative and not too large
        B_elements, B_relations = generate_boolean_algebra(n, k)

        if not B_elements or not B_relations:
            continue

        hodge_rank = hodge_diamond_rank(B_elements)
        ac0_depth = ac0_parity_depth(n, 1) # Placeholder depth; actual computation depends on

        results.append({
            "n": n,
            "k": k,
            "hodge_rank": hodge_rank,
            "ac0_depth": ac0_depth
        })

if not results:
    return {
        "metric_name": "Hodge Diamond Rank vs AC0 PARITY Depth",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_hodge_rank = sum(result["hodge_rank"] for result in results) / len(results)
mean_ac0_depth = sum(result["ac0_depth"] for result in results) / len(results)

return {
    "metric_name": "Hodge Diamond Rank vs AC0 PARITY Depth",
    "metric_value": mean_hodge_rank,
    "instances_tested": len(results),
    "conjecture_holds": mean_ac0_depth >= mean_hodge_rank,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 37)) # Default to 1

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c9efa77.py", line 83, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c9efa77.py", line 42, in run_trial
    B_elements, B_relations = generate_boolean_algebra(n, k)
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c9efa77.py", line 22, in generate_bool
    elements = [tuple(sorted(random.sample(range(2), n))) for _ in range(k)]
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 430, in sample
    raise ValueError("Sample larger than population or is negative")
ValueError: Sample larger than population or is negative

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12044
2	propose	ollama_remote	glm4:latest	0	0	11778
3	preregistration	ollama_remote	glm4:latest	0	0	6275
4	novelty	ollama_remote	glm4:latest	0	0	4781
5	novelty	ollama_remote	glm4:latest	0	0	6765
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16907
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10012

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8844
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10075
10	judge	ollama_remote	glm4:latest	0	0	10456

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97936 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9d4e7ed1496a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9d4e7ed1496a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9d4e7ed1496a.tar.gz (if generated)